Machine Learning Project

February 2026 Exam session

SDSS Classification: Analysis and Prediction

Team: Tony Dawra[1], Ilaria Mato[2], Raffaele Romano[3]

Abstract

The Sloan Digital Sky Survey (SDSS) is one of the astronomical projects that produce millions of spectral observations each year that require classification. With millions of cataloged celestial objects, manually identifying stars, galaxies, and quasars can be difficult (time consuming and expensive since it requires more people). This study explores and evaluates the performance of machine learning in this field to automating classifying celestial objects.

The dataset is available on Kaggle and it is from **SDSS Data Release 17**, made of **100,000 observations** of celestial objects. It includes a wide range of attributes, including magnitudes in the u, g, r, i, and z filters (SDSS photometric system), celestial coordinates (right ascension and declination), redshift, and observational parameters such as plate, camera column, and fiber ID. In this study, the variable `class` was chosen as the target for the multi-class classification of **GALAXY**, **QSO** (quasi-stellar object), and **STAR**.

The data was processed using KNIME Analytics, applying feature selection techniques based on Spearman correlation, the removal of identifying variables and partitioned the data using the 60/30/10 technique, respectively training, validation and test. Seven classification models were compared: Decision Tree, Naive Bayes, Random Forest, Bagging with PCA, Gradient Boosting, Multi-Layer Perceptron and Z-Score MLP. In order to evaluate the quality of the models, various metrics have been taken in consideration such as accuracy and Cohen's kappa value. The best-rated model that emerged was Bagging with PCA, achieving accuracy of **97.82%** and Cohen's Kappa of **0.9613** in the final evaluation phase on the test set.

Keywords: Machine Learning, SDSS, Stellar Classification, KNIME

[1,2,3]UniversitÃ degli Studi di Milano Bicocca, CdLM Data Science

Contents

1 Introduction **2**

2 Dataset Description **2**
- 2.1 Dataset Summary . . . 3
- 2.2 Variable Categories . . . 3

3 Preprocessing **3**
- 3.1 Workflow Overview . . . 3
- 3.2 Variable Creation . . . 3
- 3.3 Feature Engineering . . . 3
- 3.4 Data Type Transformation . . . 4
- 3.5 Missing Values . . . 4
- 3.6 Collinearity and Feature Selection . . . 5
- 3.7 Outliers and Scaling . . . 5

4 Train-Test Split and Class Balance **5**
- 4.1 Sampling Strategy . . . 6
- 4.2 Balancing / Resampling . . . 6

5 Classification Models **6**
- 5.1 Heuristic Models . . . 6
- 5.2 Probabilistic Models . . . 7
- 5.3 Separation Models . . . 7

6 Results and Model Comparison **7**
- 6.1 Evaluation Metrics . . . 7
- 6.2 Accuracy Comparison . . . 7
- 6.3 Best Model . . . 7
- 6.4 Interpretation and Discussion . . . 8
- 6.5 Confusion Matrix Analysis . . . 8

7 Redshift Impact Analysis **8**

8 ROC-AUC Analysis **9**
- 8.1 With Redshift Feature . . . 9
- 8.2 Without Redshift Feature . . . 9
- 8.3 AUC Comparison Summary . . . 9

9 Conclusion and Recommendations **10**

A Variables Description **10**

1 Introduction

Classifying astronomical objects represents a fundamental challenge for modern astrophysics, understanding the formation of galaxies and the large-scale structure of the universe. With the advent of the Sloan Digital Sky Survey (SDSS) [1], we have managed to catalog over 500 million celestial objects and produced more than 4 million optical spectra, increasing the volume of astronomical data and surpassing manual classification by astronomers [2].

Research in observational astronomy has demonstrated that accurately identifying stars, galaxies, and quasars is crucial. Studies have shown that distinguishing between these classes of objects allows researchers to map the distribution of matter in the universe.

Furthermore, misclassification of astronomical objects can lead to significant biases in statistical analyses, impacting conclusions about fundamental cosmological parameters such as the Hubble constant, the distribution of dark matter, and the rate of cosmic expansion.

The goal of this study is to develop and evaluate machine learning models capable of automatically classifying astronomical objects into three main categories: GALAXY, QSO (quasi-stellar object or quasar), and STAR, based on photometric measurements and celestial coordinates. By exploiting measurable properties such as the five-band photometric system (u, g, r, i, z filters), redshift values, and positional data (right ascension and declination), we aim to build a classification framework that achieves high accuracy while maintaining interpretability and reproducibility.

2 Dataset Description

The Stellar Classification Dataset (SDSS17) from Kaggle is based on observations from the Sloan Digital Sky Survey Data Release 17, one

of the most comprehensive astronomical surveys ever conducted. The dataset contains approximately 100,000 celestial object observations, each characterized by photometric, positional, and spectroscopic measurements.

As shown in Table 11 , the distribution of features across celestial object classes reveals the structure and variability of the data.

2.1 Dataset Summary

- **Rows:** ∼100,000 observations
- **Columns:** 18 total (17 predictors + 1 target)
- **Target column:** `class` (GALAXY, QSO, STAR)
- **Final features:** ∼7 after preprocessing

2.2 Variable Categories

- **Positional/Astrometric:** alpha, delta
- **Photometric System:** u, g, r, i, z
- **Physical/Spectroscopic:** redshift, class
- **Observational Parameters:** plate, MJD, cam_col
- **Identifiers (removed):** obj_ID, spec_obj_ID, fiber_ID, field_ID, run_ID, rerun_ID

Identifier-like columns were removed before training to avoid data leakage and to ensure models focus on the physical measurements rather than record-specific IDs.

3 Preprocessing

3.1 Workflow Overview

The first step to work in KNIME is the data import, in this specific case the CSV reader node. After loading the file it is common to verify if there are any missing values. In this dataset there were no missing values so no further action was needed. The next step is to remove the columns which do not bring value to the process the columns removed were ID-type columns.

After this first part, it was mandatory to explore the correlation between features. Highly correlated features do not bring value to the classification algorithms since they are just repetition of the same data, which can even lead to worse predictions.

Once the data are ready, the next step is dividing the dataset in three parts: train, validation and test (Out Of Sample) in the following proportion: **60/30/10**. In the Table Partitioner node, stratified sampling was set to keep the same distribution of the original dataset and the same seed (71) to maintain consistency.

3.2 Variable Creation

The only additional variable introduced was a random seed (set to 71) to ensure reproducibility of the train-test split.

3.3 Feature Engineering

Motivation. During initial experiments, we observed that models trained exclusively on the original photometric magnitudes (u, g, r, i, z) achieved unsatisfactory accuracy—particularly in distinguishing quasars from galaxies, which share similar broadband colors. The redshift feature proved to be the most discriminative variable; however, to reduce over-reliance on a single feature and to improve model robustness, we engineered three additional variables based on well-established practices in astronomical classification [2].

In astrophysics, *color indices* (magnitude differences between bands) and *flux ratios* are standard tools for characterizing stellar populations and distinguishing between different

types of celestial objects. These derived features encode information about the spectral energy distribution (SED) of an object, which directly relates to its physical properties such as temperature, composition, and redshift.

The three engineered features are:

1. **flux_i_z** — Flux ratio between the i and z bands:

$$\texttt{flux_i_z} = 10^{-0.4 \cdot (i-z)}$$

   This captures the relative brightness in the near-infrared spectrum. The difference $(i - z)$ is a color index; converting to flux ratio emphasizes physical brightness.

2. **color_span_norm** — Normalized color span:

$$\texttt{color_span_norm} = \frac{u - z}{\bar{m}}$$

   where $\bar{m} = \text{mean}(u, g, r, i, z)$. This measures the spectral width normalized by average magnitude. Positive values indicate red objects, negative values blue.

3. **spectral_index** — Spectral slope indicator:

$$\texttt{spectral_index} = \log_{10}(F_{ug}) - \log_{10}(F_{iz})$$

   where $F_{ug} = 10^{-0.4(u-g)}$ and $F_{iz} = 10^{-0.4(i-z)}$. This compares the SED slope in UV vs infrared regions. Quasars exhibit distinct spectral indices, aiding QSO identification.

The following Python code was used in KNIME's Python Script node:

```python
import knime.scripting.io as knio
import numpy as np

# Read input
df = knio.input_tables[0].to_pandas()

# 1. flux_i_z
df['flux_i_z'] = 10**(-0.4 * (df['i'] - df['z']))

# 2. color_span_norm
df['color_span_norm'] = (df['u'] - df['z']) / df[['u','g','r','i','z']].mean(axis=1)

# 3. spectral_index
flux_u_g = 10**(-0.4 * (df['u'] - df['g']))
df['spectral_index'] = np.log10(flux_u_g + 1e-10) - np.log10(df['flux_i_z'] + 1e-10)

# Handle inf/nan
for col in ['flux_i_z', 'color_span_norm', 'spectral_index']:
    df[col] = df[col].replace([np.inf, -np.inf], np.nan).fillna(0)

# Output
knio.output_tables[0] = knio.Table.from_pandas(df)
```

These engineered features, combined with the original photometric data and the crucial redshift variable, significantly improved classification performance—especially for the challenging QSO class. The approach demonstrates how domain knowledge from astrophysics can guide effective feature engineering in machine learning pipelines.

3.4 Data Type Transformation

Most of the dataset columns are numeric and were treated as continuous features. The target variable (`class`) was treated as categorical for multi-class classification.

3.5 Missing Values

After importing and performing basic checks, we observed no missing values in the working dataset, so no imputation or deletion of rows was necessary.

3.6 Collinearity and Feature Selection

To reduce redundancy and improve model stability, we analysed collinearity among numeric predictors using a Spearman rank correlation matrix (appropriate for non-normal distributions). We applied a correlation filter with a threshold of 0.7 to remove strongly correlated features.

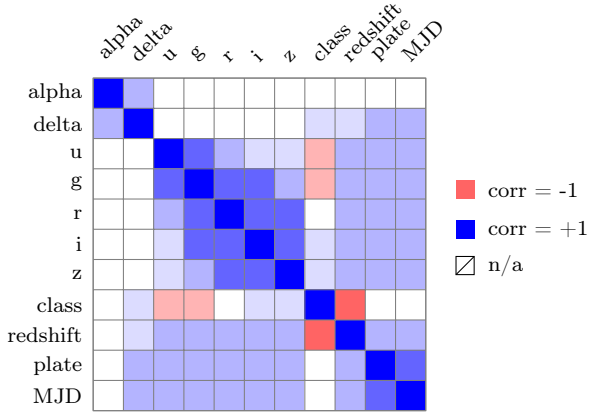


Table 1: Spearman correlation matrix before feature selection (original features only, without engineered features).

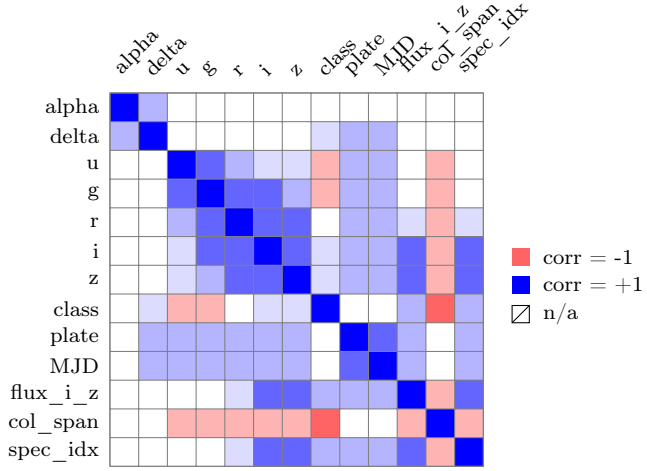


Table 2: Spearman correlation matrix before feature selection, including engineered features (flux_i_z, color_span_norm, spectral_index).

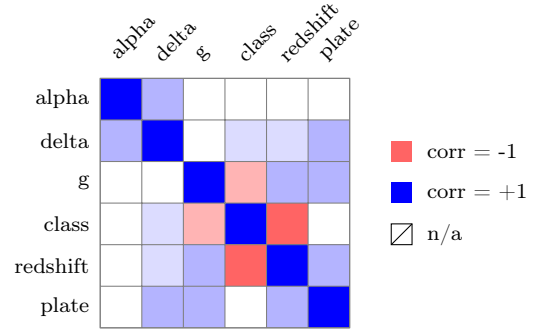


Table 3: Spearman correlation matrix after feature selection (original features only, without engineered features).

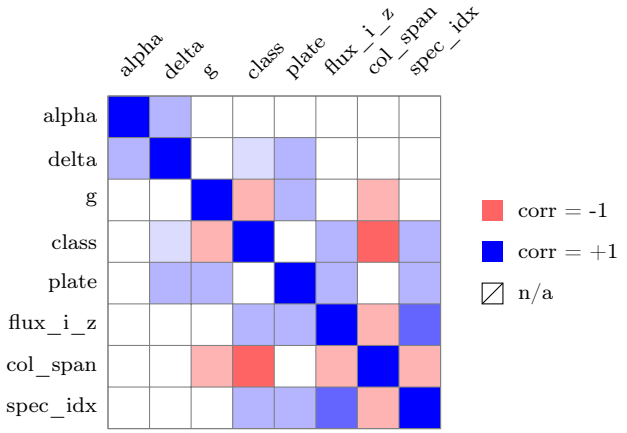


Table 4: Spearman correlation matrix after feature selection, including engineered features (flux_i_z, color_span_norm, spectral_index).

3.7 Outliers and Scaling

We did not apply outlier removal because the distant points represent valid astronomical measurements of celestial body distances. However, scaling was important for the MLP branch: we standardized numeric predictors using Z-score normalization so each input feature was on a comparable scale.

4 Train-Test Split and Class Balance

To estimate generalization on unseen data, we divided the data into 60% training, 30% val-

idation, and 10% test. The split was created with KNIME's Table Partitioner node using a fixed random seed (71) for reproducibility. We also enabled stratified sampling on the `class` column.

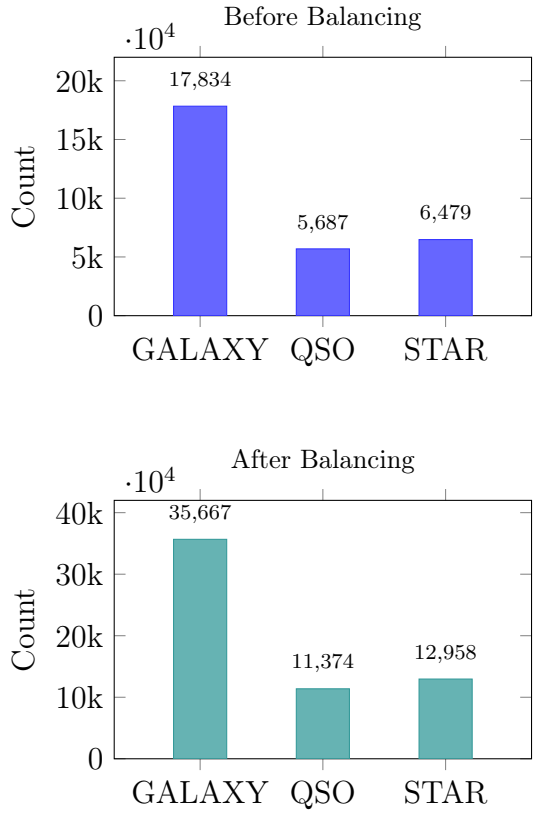


Figure 1: Class distribution before (top) and after (bottom) balancing.

4.1 Sampling Strategy

Stratification is particularly important in multi-class problems: if one class is underrepresented in the test set, accuracy can look artificially high (or low) and confusion matrices become harder to interpret. The 60/30/10 split gave us a large training set while still leaving sizeable validation and test sets for reliable comparisons.

4.2 Balancing / Resampling

We did not apply explicit resampling methods such as SMOTE or undersampling. Based on the class counts after the stratified split, each class remained well represented, so we trained directly on the original training partition.

5 Classification Models

Various Machine Learning classification techniques were applied to predict celestial object classes (stars, galaxies, and quasars) based on spectroscopic observations from the Sloan Digital Sky Survey. These models were grouped into different categories based on their underlying approach. To train the models, the Learner Node was utilized; it handles the model training phase and is specific to each type of machine learning algorithm. It is designed to build the model based on the provided training data. Predictions were then made using the Predictor Node, which enables a pretrained machine learning model to be used on the test dataset to generate predictions. The model's performance was evaluated with the Scorer Node, which is used to evaluate the performance of a machine learning model after it has been trained and applied to a dataset.

5.1 Heuristic Models

These models utilize rule-based decision-making and partition the dataset through tree structures.

- **Decision Trees:** employed to develop interpretable models that predict celestial object classes based on spectroscopic attributes. These models use a hierarchical structure, where internal nodes represent decisions based on specific features, and leaf nodes predict class labels.
  - **Random Forest:** an ensemble learning method that builds multiple decision trees and combines their predictions for improved accuracy and robustness, making it less prone to

overfitting than individual decision trees.

- **Ensemble Trees:** combine multiple models to improve classification performance, aiming to reduce overfitting, increase stability, and enhance generalization.
  - **Gradient Boosting:** sequentially builds models to correct errors made by previous models, offering high accuracy but requiring careful tuning to avoid overfitting.
  - **Bagging:** trains multiple models on different data subsets and combines their predictions to reduce variance and overfitting.

5.2 Probabilistic Models

- **Naive Bayes:** a probabilistic classifier based on Bayes' theorem, assuming feature independence. Both Naive Bayes and Naive Bayes Learner Nodes were employed in training to classify celestial objects based on their spectroscopic features.

5.3 Separation Models

These models aim to partition or separate the feature space into different regions or classes by finding an optimal decision boundary.

- **Artificial Neural Networks:** computational models consisting of layers of interconnected nodes. Each node processes inputs, applies weights, and passes the output through an activation function to the next layer.
  - **Multilayer Perceptron (MLP):** consists of multiple layers of neurons, including at least one hidden layer, where each neuron is connected to others via weighted connections. The model's ability to learn from multiple layers of features and refine its predictions makes it a powerful tool for improving classification accuracy and identifying the spectroscopic patterns that distinguish stars, galaxies, and quasars.

6 Results and Model Comparison

6.1 Evaluation Metrics

We compared models using two metrics computed on the held-out test set:

- **Accuracy:** the fraction of correctly classified instances.
- **Cohen's kappa (κ):** agreement corrected for chance, ranging from 0 (random) to 1 (perfect agreement).

6.2 Accuracy Comparison

Heuristic Models

The accuracy of different tree-based models was compared. Among Decision Trees, the standard Decision Tree achieved the highest accuracy. For Ensemble methods, Bagging with PCA outperformed all other models.

	Model	Acc.
Decision Trees	Decision Tree	0.977
	Random Forest	0.972
Ensemble Trees	Bagging + PCA	0.978
	Gradient Boosting	0.967

Table 5: Heuristic Models accuracy.

6.3 Best Model

After identifying the best models within each family, a final comparison was performed.

Model	Acc. (%)	κ
Bagging + PCA	97.82	0.9613
Decision Tree	97.69	0.9588
Random Forest	97.23	0.9505
Gradient Boosting	96.69	0.9411
MLP (MinMax)	95.56	0.9211
MLP (Z-Score)	93.08	0.8765
Naive Bayes	75.00	0.5958

Table 6: Model comparison on test set (sorted by accuracy).

6.4 Interpretation and Discussion

Across the tested approaches, ensemble methods came out on top. **Bagging with PCA** achieved the highest test performance (Accuracy = 97.82%, $\kappa = 0.9613$), suggesting that the combination of bootstrap aggregation with dimensionality reduction effectively captured the main patterns while reducing noise and overfitting.

Decision Tree and Random Forest were very close behind, demonstrating that tree-based methods are particularly well-suited for this classification task. Gradient Boosting also performed strongly but did not surpass the Bagging approach.

The MLP models produced good results, with the MinMax-normalized version (95.56%) outperforming the Z-score normalized version (93.08%), indicating that the choice of normalization strategy impacts neural network performance.

Naive Bayes was the weakest model (Accuracy = 75.00%, $\kappa = 0.596$). This result makes sense because Naive Bayes assumes conditional independence between predictors, and photometric bands tend to be correlated even after feature filtering.

6.5 Confusion Matrix Analysis

To understand the misclassification patterns, we compare the confusion matrices for the best (Bagging + PCA) and worst (Naive Bayes) models on the validation set (30,000 samples).

Bagging + PCA (Validation Set):

True / Pred	GAL	QSO	STAR
GALAXY	**17,590**	216	28
QSO	407	**5,279**	1
STAR	1	0	**6,478**

Table 7: Bagging + PCA confusion matrix: 653 errors (2.18%).

Naive Bayes (Validation Set):

True / Pred	GAL	QSO	STAR
GALAXY	**12,530**	5,160	144
QSO	247	**5,437**	3
STAR	1,136	810	**4,533**

Table 8: Naive Bayes confusion matrix: 7,500 errors (25%).

Key Observations:

- **STAR classification:** Both models classify STAR well, but Bagging is near-perfect (only 1 error) while Naive Bayes misclassifies 1,946 STARs.
- **GALAXY↔QSO confusion:** This is the main source of errors for both models. Naive Bayes classifies 5,160 GALAXYs as QSO (29% error rate), while Bagging only misclassifies 216 (1.2%).
- **Physical explanation:** GALAXYs and QSOs have similar photometric signatures, making them harder to distinguish without redshift information.

7 Redshift Impact Analysis

A key finding of this study was the critical importance of the `redshift` feature for accurate classification. We conducted an ablation study by removing the redshift variable and retraining the models to observe the impact on classification performance.

Methodology

We selected two representative models for this comparison:

- **Bagging + PCA:** the best-performing model
- **Naive Bayes:** the worst-performing model

Both models were retrained on the same dataset with redshift excluded from the feature set.

Results Without Redshift

Model	Validation		Test	
	Acc.	κ	**Acc.**	κ
Bagging + PCA	87.0%	0.77	87.0%	0.77
Naive Bayes	34.0%	0.08	35.0%	0.08

Table 9: Model performance without redshift feature.

Discussion

Removing redshift caused a significant drop in classification accuracy for both models. This demonstrates that redshift is a highly discriminative feature for distinguishing between GALAXY, QSO, and STAR classes:

- **QSOs (Quasars)** typically have high redshift values ($z > 0.1$) due to their cosmological distances
- **Stars** have near-zero redshift as they are within our galaxy
- **Galaxies** have intermediate redshift values

This physical relationship makes redshift one of the most important predictors for celestial object classification, and its removal forces models to rely solely on photometric features, which are less discriminative.

8 ROC-AUC Analysis

To provide a more comprehensive evaluation beyond accuracy and kappa, we computed ROC (Receiver Operating Characteristic) curves and AUC (Area Under the Curve) scores for each class. This analysis compares the best model (Bagging + PCA) with the worst model (Naive Bayes) to illustrate the performance gap.

8.1 With Redshift Feature

Figure 2 shows the ROC curves for the Bagging + PCA model on the validation set with redshift included. The model achieves near-perfect AUC scores ($\approx$ 0.99) for all three classes, demonstrating excellent discrimination capability.

Figure 3 presents the ROC curves for the Naive Bayes model under the same conditions. The model shows lower AUC values, particularly for the GALAXY class, reflecting its weaker performance.

8.2 Without Redshift Feature

Figure 4 illustrates the impact of removing redshift on the Bagging + PCA model. While still performing reasonably well, there is a noticeable degradation in AUC, especially for the QSO class.

Figure 5 shows the ROC curves for Naive Bayes without redshift. The performance drop is more severe, confirming that Naive Bayes relies heavily on redshift for accurate classification.

8.3 AUC Comparison Summary

The ROC curves clearly demonstrate:

1. **Bagging + PCA** achieves near-perfect AUC scores ($\approx$ 0.99) for all three classes when redshift is included.
2. **Naive Bayes** shows lower AUC values, particularly for the GALAXY class.
3. Removing redshift degrades AUC for both models, but the impact is more severe for distinguishing QSOs.

9 Conclusion and Recommendations

We implemented an end-to-end supervised learning workflow in KNIME to classify astronomical objects using a consistent preprocessing and evaluation framework. After removing identifier fields, reducing correlated predictors, and creating a stratified 60/30/10 hold-out split, we trained and compared multiple classifiers.

Key Findings

1. Best Model: Bagging with PCA achieved the highest performance (Accuracy = 97.82%, $\kappa = 0.9613$), outperforming Random Forest and other ensemble methods.

2. Importance of Redshift: The redshift feature is critical for accurate classification. Removing it causes significant performance degradation across all models, as it provides direct physical information about the distance and nature of celestial objects.

3. Model Suitability: Tree-based ensembles (Bagging, Decision Tree, Random Forest) are particularly well-suited for this task due to their ability to capture non-linear relationships in photometric data.

4. Naive Bayes Limitations: The poor performance of Naive Bayes confirms that the independence assumption is violated by the correlated photometric features.

References

[1] fedesoriano. (January 2022). Stellar Classification Dataset – SDSS17. Kaggle. Retrieved from `https://www.kaggle.com/fedesoriano/stellar-classification-dataset-sdss17`

[2] Sloan Digital Sky Survey. SDSS Data Release 17. Retrieved from `https://www.sdss.org/dr17/`

[3] KNIME AG. KNIME Analytics Platform. Retrieved from `https://www.knime.com/`

A Variables Description

Table 10: Complete list of SDSS17 dataset variables

Variable	Description
obj_ID	Object Identifier, unique value in the image catalog
alpha	Right Ascension angle (at J2000 epoch)
delta	Declination angle (at J2000 epoch)
u	Ultraviolet filter in the photometric system
g	Green filter in the photometric system
r	Red filter in the photometric system
i	Near Infrared filter in the photometric system
z	Infrared filter in the photometric system
run_ID	Run Number for the specific scan
rerun_ID	Rerun Number for image processing specification
cam_col	Camera column (scanline identifier)
field_ID	Field number identifier
spec_obj_ID	Unique ID for optical spectroscopic objects
class	Object class (GALAXY, STAR, or QSO)
redshift	Redshift value (wavelength increase measure)
plate	Plate ID in SDSS
MJD	Modified Julian Date of observation
fiber_ID	Fiber that pointed light at the focal plane

Table 11: Dataset statistics overview (18 variables, 100,000 observations).

Variable	Type	Missing	Unique	Min	Max	Mean	Std Dev	Median	Most Common
alpha	Float	0	99,999	0.006	360	177.63	96.50	180.90	34.75 (2; 0.0%)
delta	Float	0	99,999	-18.79	83.00	24.16	19.65	23.65	-0.60 (2; 0.0%)
u	Float	0	93,748	-9,999	32.78	21.18	31.76	22.18	24.64 (77; 0.08%)
g	Float	0	92,651	-9,999	31.60	20.53	31.75	21.1	25.11 (9; 0.01%)
r	Float	0	91,901	9.82	29.97	20.05	1.86	20.13	24.80 (21; 0.02%)
i	Float	0	92,019	9.47	32.14	19.09	1.76	19.41	24.36 (6; 0.01%)
z	Float	0	92,007	-9,999	29.38	18.87	31.73	19.01	22.83 (58; 0.06%)
class	String	0	3	–	–	–	–	–	GALAXY (59,445; 59.4%)
redshift	Float	0	99,295	-0.01	7.01	0.58	0.73	0.42	0 (412; 0.41%)
plate	Integer	0	6,284	266	12,547	5,137	2,491	4,987	6,301 (98; 0.1%)
MJD	Integer	0	2,180	51,608	58,932	55,589	1,808	55,869	56,625 (249; 0.25%)
fiber_ID	Integer	0	1,000	1	1,000	449	273	433	637 (159; 0.16%)
cam_col	Integer	0	6	1	6	3.51	1.59	4	4 (19,573; 19.6%)
field_ID	Integer	0	856	11	999	186	149	146	71 (472; 0.47%)
run_ID	Integer	0	430	109	8,162	4,481	1,995	4,188	3,699 (2,450; 2.5%)
rerun_ID	Integer	0	1	301	301	301	0	301	301 (100,000; 100%)
obj_ID	Float	0	80,846	1.24×10^{18}	1.24×10^{18}	1.24×10^{18}	8.44×10^{15}	1.24×10^{18}	–
spec_obj_ID	Float	0	100,000	2.99×10^{17}	1.41×10^{19}	2.80×10^{18}	3.32×10^{18}	5.61×10^{17}	–

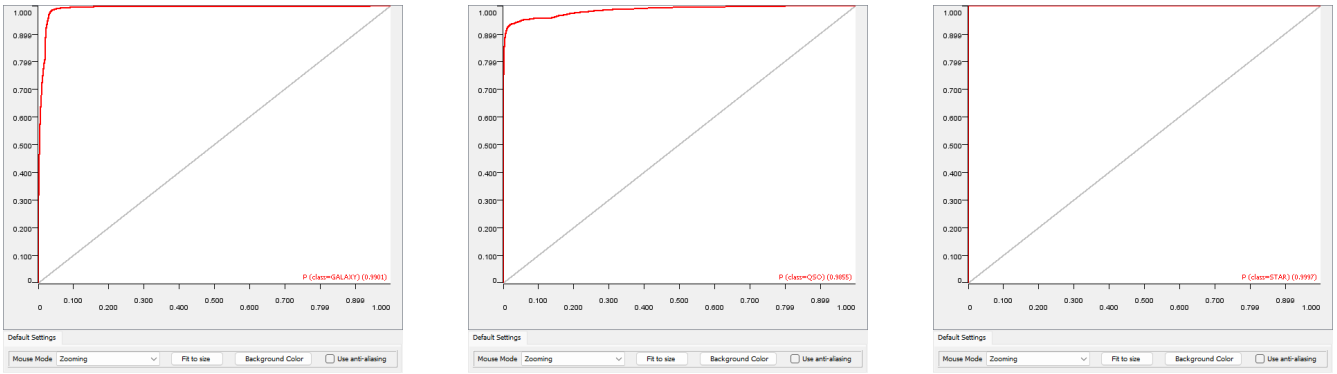


Figure 2: ROC curves for Bagging + PCA model (validation set, with redshift).

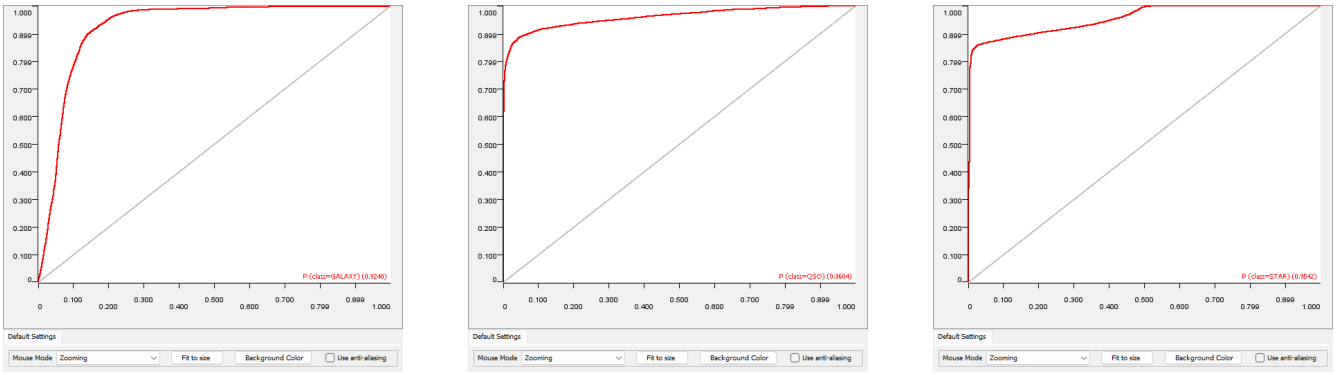


Figure 3: ROC curves for Naive Bayes model (validation set, with redshift).

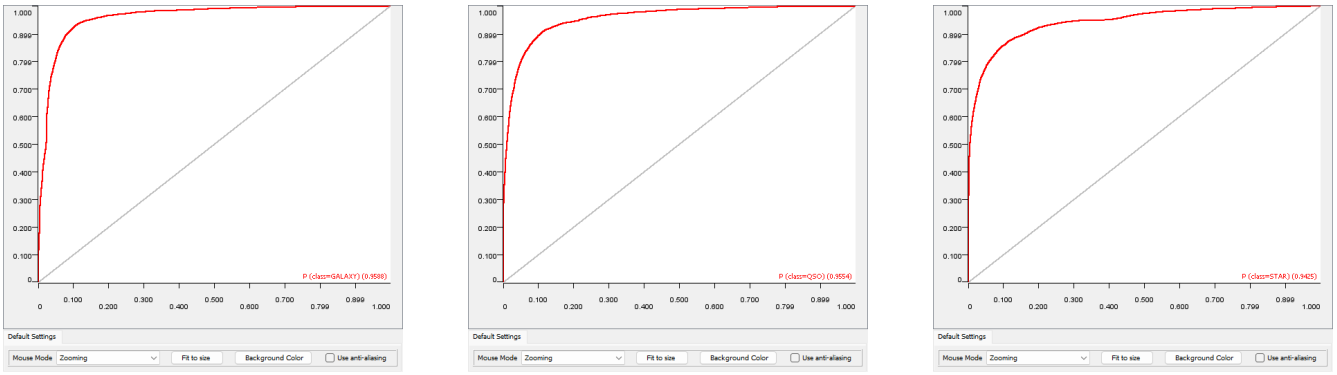


Figure 4: ROC curves for Bagging + PCA model without redshift feature.

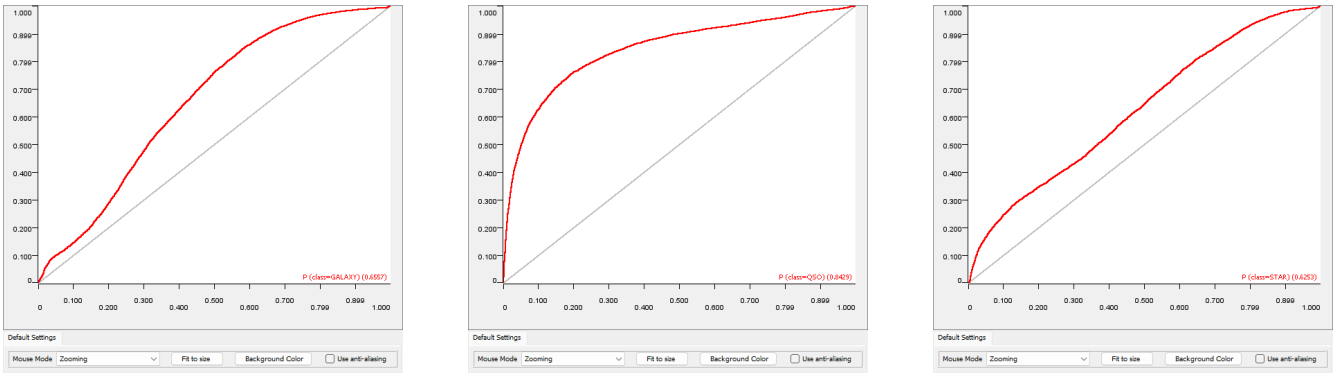


Figure 5: ROC curves for Naive Bayes model without redshift feature.